

Analisi delle performance di 3 modelli di machine learning su un dataset progressivamente sporcato

Andrea Spagnolo 879254

Introduzione

Introduzione

Processo di degradazione dataset

Valori nulli

Implementazione

Analisi risultati

valori duplicati

implementazione

Analisi risultati

outliers

Implementazione

Analisi risultati

rumore su feature target

implementazione

Analisi risultati

Conclusioni

Codice Sorgente

L'obiettivo di questo progetto è stato analizzare l'impatto di diverse tipologie di errori sulla qualità di un dataset. Ho voluto studiare in che modo la degradazione progressiva dei dati influenzi le performance di tre modelli di machine learning distinti: **Decision Tree**, **Support Vector Machine (SVM)** e **Neural Network**.

Per ogni fase di degradazione, ho addestrato e valutato ciascuno dei tre modelli sia nella loro versione **naïve** che in quella **ottimizzata**. Questa doppia valutazione mi ha permesso di capire meglio se e come l'ottimizzazione dei parametri possa mitigare gli effetti negativi degli errori presenti nel dataset.

La mia analisi si è concentrata sul monitoraggio delle metriche di **accuracy**, **precision**, **recall**, **F1-score** e **delle matrici di confusione**, man mano che la qualità del dataset diminuiva. L'obiettivo principale era studiare l'evoluzione delle performance dei modelli al peggiorare della qualità del dataset. Quello che mi aspetto è di osservare una correlazione

diretta tra l'aumento degli errori e un calo delle performance, ma con variazioni significative tra i diversi modelli e le loro versioni.

Processo di degradazione dataset

Per il processo di degradazione della qualità del dataset sono stati usati diversi approcci:

1. valori nulli
2. valori duplicati
3. outliers
4. rumore sulla variabile target

Per ognuno di questi approcci è stata utilizzata la seguente pipeline di lavoro:

1. Richiamo dal dataset preprocessato
2. Divisione in feature binarie/categoriche e continue
3. Calcolo mutua informazione feature su dataset pulito
4. Applicazione procedura di degradazione dataset e calcolo mutua informazione delle feature per ogni step di degradazione del dataset
5. Visualizzazione dataset degradato
6. Addestramento modelli su ogni step di degradazione del dataset e calcolo delle relative metriche (accuracy, recall ...) e matrici di confusione
7. Visualizzazione andamento metriche al crescere del livello di degradazione del dataset
8. Visualizzazione andamento matrici di confusione al crescere del livello di degradazione del dataset
9. Analisi sull'evoluzione dell'importanza delle feature nel decision tree al crescere del livello di degradazione delle feature
10. Analisi sull'evoluzione della mutua informazione al crescere dello sporcamento del dataset

Valori nulli

La presenza di valori mancanti nei dataset è un problema estremamente comune e pervasivo nel mondo reale. Le ragioni sono molteplici: guasti dei sensori, errori di inserimento dati, sondaggi a cui non si risponde completamente ...

Quindi essendo un problema molto diffuso analizzare come i valori nulli vadano ad impattare sulle performance risulta molto utile in quanto ci permette di valutare quali sono i **modelli più robusti** a questo tipo di problema e quali sono i **metodi di imputazione più efficaci** da utilizzare. Inoltre, l'approccio di degradazione a step ci consente di identificare il **punto di rottura** del modello, ovvero la percentuale di dati mancanti oltre la quale le sue performance iniziano a crollare drasticamente. Conoscere questo valore è fondamentale in quanto ci fornisce una soglia di accettabilità per la qualità dei dati, aiutandoci a scartare i dataset non sufficientemente "solidi" per un'analisi significativa.

Implementazione

Per implementare questo processo di degradazione abbiamo utilizzato la pipeline precedentemente descritta, ma con dei passaggi aggiuntivi dovuti all'analisi del migliore metodo di imputazione per ogni modello.

Di seguito la funzione utilizzata per inserire progressivamente i valori mancanti nel dataset

```
def introduce_missing_values(df: pd.DataFrame, columns: list, percentage: float) -> pd.DataFrame:

    df_degraded = df.copy()
    for col in columns:
        non_missing_indices = df_degraded[col].dropna().index
        n_to_make_missing = int(len(non_missing_indices) * percentage)

        missing_indices = np.random.choice(
            non_missing_indices,
            size=n_to_make_missing,
            replace=False
        )
        df_degraded.loc[missing_indices, col] = np.nan

    return df_degraded
```

Questa funzione è stata applicata a tutto il dataset esclusa la colonna target in quanto utilizzando tutti algoritmi di apprendimento supervisionato è necessario avere la variabile target presente per tutte le istanze del dataset

```
features_to_degrade = df.columns.drop("HeartDisease")
```

La funzione è poi stata lanciata per ogni step di degradazione, ovvero da 0 a 90 con intervalli di 10.

```
for p in missing_percentages:
    print(f"Generando dataset con {int(p*100)}% di valori mancanti...")
    degraded_datasets[p] = introduce_missing_values(
        df=df,
        columns=features_to_degrade,
        percentage=p
    )
```

Una volta ottenuti i vari dataset degradati in maniera progressiva sono stati addestrati i vari modelli (decision tree, svm e neural networks) sia nella loro versione naive che ottimizzata. Inoltre per ognuno di questi modelli sono stati provati diversi metodi di imputazione per studiare il più efficace. In particolare abbiamo sostituito i dati mancanti utilizzando la media, la mediana, la moda e come metodologie più sofisticate abbiamo provato knn e il metodo iterativo.

```
imputation_strategies = {
    'mean': SimpleImputer(strategy='mean'),
    'median': SimpleImputer(strategy='median'),
    'mode': SimpleImputer(strategy='most_frequent'),
    'knn': KNNImputer(n_neighbors=5),
```

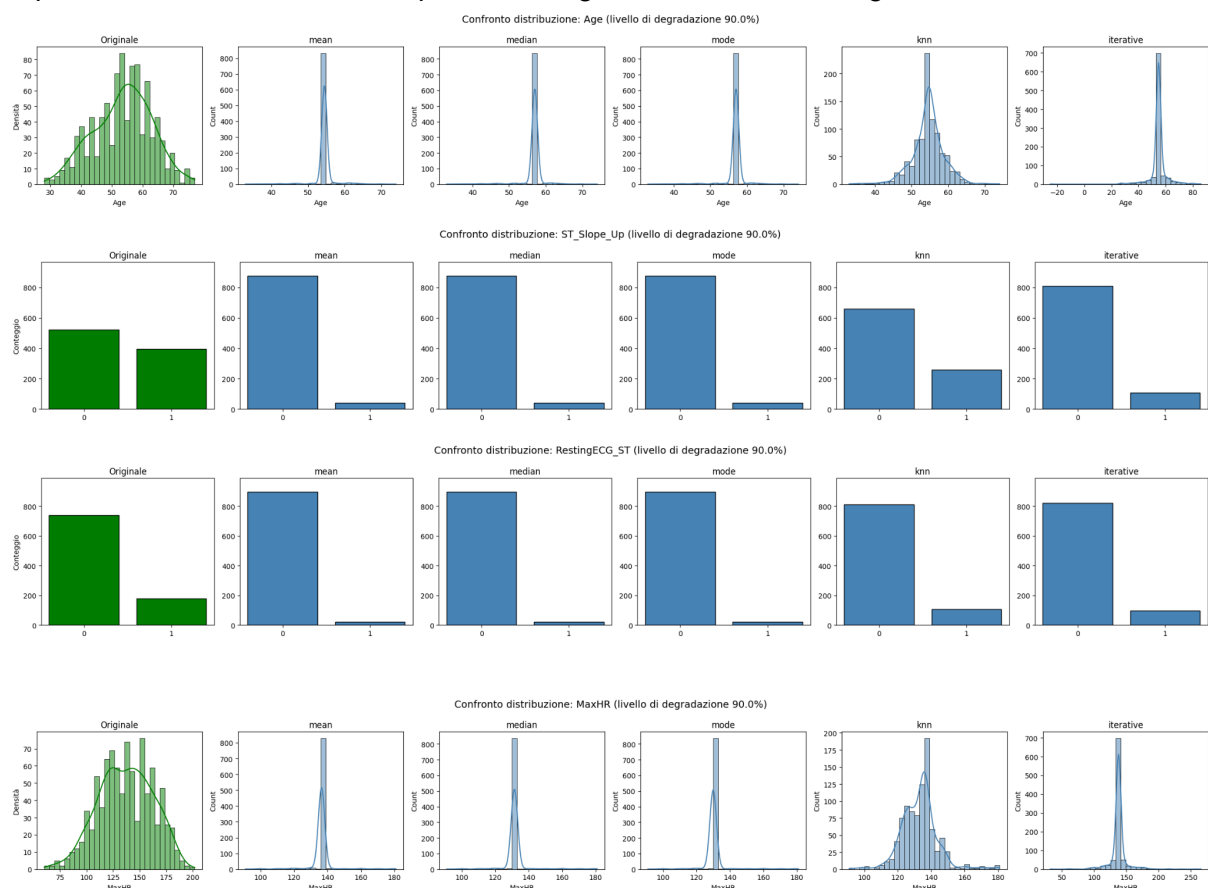
```
'iterative': IterativeImputer(random_state=42)
}
```

Una volta completata l'imputazione dei valori mancanti, mi sono assicurato della validità dei dati prodotti. Ho riscontrato che, per le feature binarie, i metodi di imputazione generavano valori non binari (ad esempio, 0.2, 0.5, 0.8), che non erano coerenti con il dominio del dato (0 o 1). Per correggere questo problema e garantire l'integrità del dataset, ho applicato una soglia di discretizzazione. Il codice riportato di seguito rende i valori binari, arrotondando a 1 quelli superiori a 0.5 e a 0 quelli inferiori o uguali a 0.5

```
for col in binary_features:
    if col in X_train_imp_df.columns:
        X_train_imp_df[col] = (X_train_imp_df[col] > 0.5).astype(int)
        X_test_imp_df[col] = (X_test_imp_df[col] > 0.5).astype(int)
```

Analisi risultati

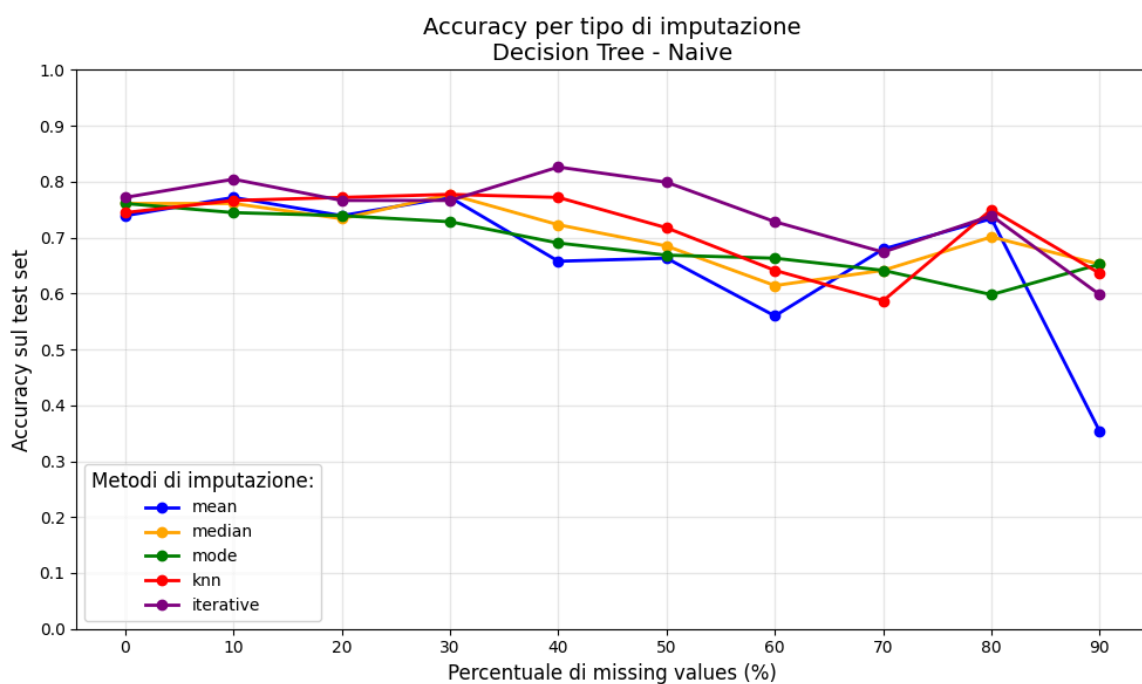
Come prima cosa ho ritenuto utile andare a rappresentare la distribuzione ottenuta alla fine del procedimento di degradazione del dataset, per osservare anche quali tra i metodi di imputazione usati andavano a riprodurre meglio la distribuzione originale.

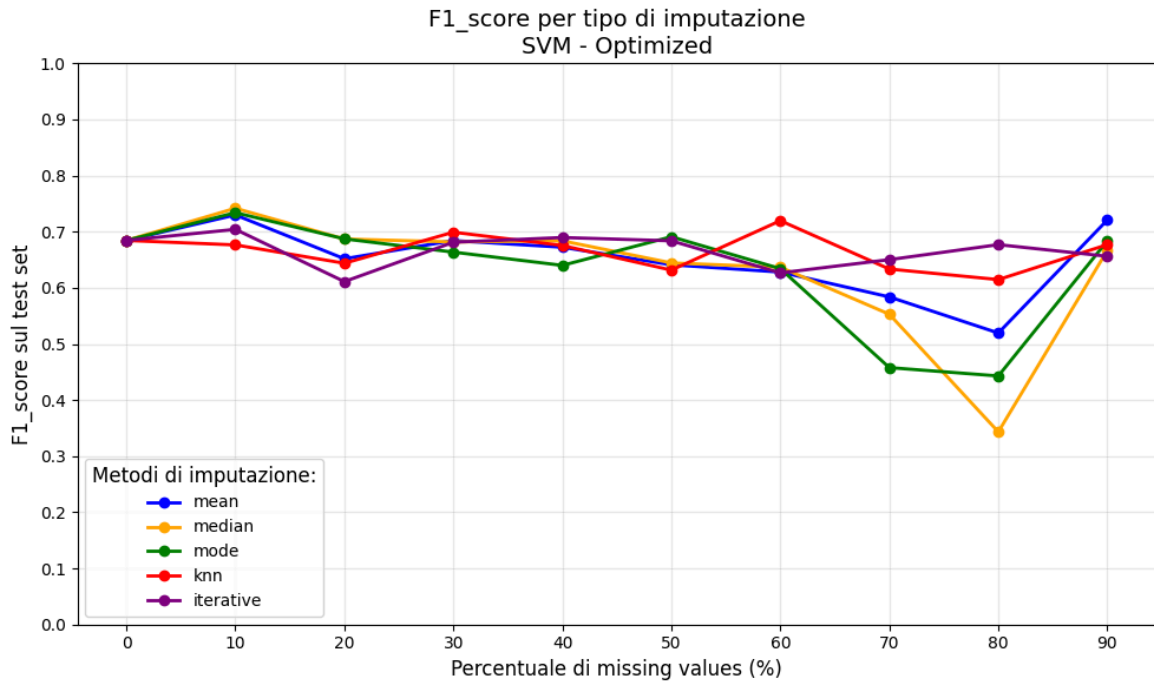


Per osservare i grafici delle distribuzioni di tutte le feature guardare nel file [missing_values](#) su [github](#).

I grafici mostrano delle distribuzioni fortemente compresse, sintomo che la nostra funzione di degradazione ha lavorato correttamente. Le distribuzioni più compresse risultano essere quelle continue in cui solo il metodo di imputazione knn sembra restituire una distribuzione minimamente paragonabile. Anche le feature binarie risultano compromesse con metodi di imputazione “leggeri” come media, moda e mediana, mentre con knn e il metodo iterativo si mantengono più simili alla distribuzione originale. Da questa prima analisi risulta quindi che i metodi knn e il metodo iterativo, seppur molto più onerosi a livello computazionale, sembrano essere l'unica scelta praticabile se si ha una percentuale di valori mancanti molto alta.

Per analizzare i risultati ho creato dei grafici che mostrano l'andamento di ogni singola metrica per ogni modello utilizzato in modo tale da analizzare il comportamento di quella singola metrica e inoltre studiare quale fosse il migliore metodo di imputazione. Di seguito riporto qualche grafico di esempio:

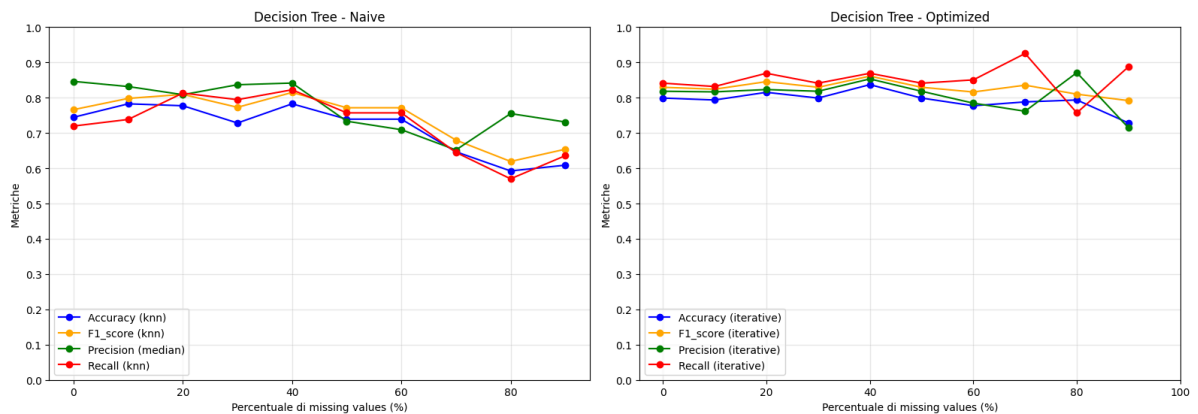


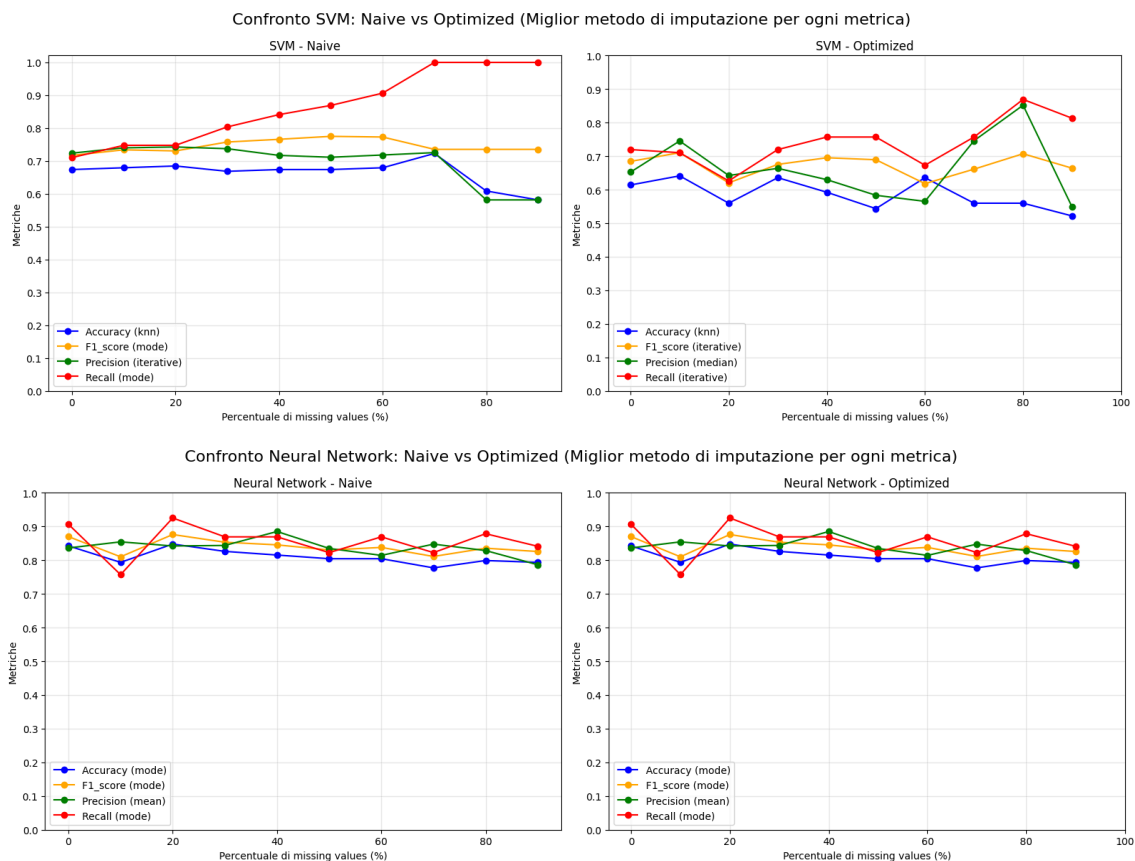


Questa funzione andava a produrre 4 grafici (uno per metrica) per ogni singolo modello per ogni singolo approccio per un totale di 24 grafici il che rende poca immediata la comprensione dei risultati. A fronte di ciò ho prodotto dei nuovi grafici in cui andavo a selezionare il metodo di imputazione che mediamente aveva registrato le performance migliori in questo modo riuscivo a mostrare l'andamento di tutte e 4 le metriche in un singolo grafico facendo scendere il numero di grafici prodotti da 24 a soli 6 grafici

```
avg_by_imp = approach_df.groupby('imputation')[metric].mean()
best_imp = avg_by_imp.idxmax()
best_df = approach_df[approach_df['imputation'] ==
best_imp].sort_values('missing_rate')
```

Confronto Decision Tree: Naive vs Optimized (Miglior metodo di imputazione per ogni metrica)





I grafici mostrano come il modello che ha registrato performance più solide è stato senza dubbio la rete neurale in cui si ha solo un leggero calo di performance. SVM, nella sua versione naive, ha riscontrato delle performance accettabili fino al 60% di valori mancanti, mentre successivamente le performance crollano e il modello tende a classificare tutte le istanze come positive, come si può osservare dall'impennata del valore di recall a fronte della discesa dell'accuracy. La SVM ottimizzata invece non ha performance ottime, ma rimangono comunque molto più stabili rispetto al suo corrispettivo naive. L'albero di decisione invece ha riscontrato un comportamento abbastanza allineato per tutte le metriche con un leggero calo progressivo.

Un'altra osservazione importante è che i modelli ottimizzati, come prevedibile, hanno mostrato performance nettamente migliori rispetto ai loro corrispettivi naive, con andamenti molto più stabili delle metriche. Questo trend, che vedremo si ripeterà anche per tutte le altre analisi, può essere spiegato dal fatto che modelli di machine learning ottimizzati hanno una maggiore capacità di gestione del rumore e delle imperfezioni nei dati, a differenza dei modelli naive, che spesso sono sbilanciati e vulnerabili a piccole variazioni del dataset, i modelli ottimizzati trovano il giusto equilibrio tra **bias** e **varianza**. Questo significa che sono meno inclini a overfitting e a diventare ipersensibili ai singoli dati corrotti. In pratica, mentre un modello naive costruisce i suoi confini decisionali senza alcuna difesa contro il rumore, un modello ottimizzato utilizza i suoi iperparametri, come la profondità massima di un albero decisionale o i fattori di regolarizzazione, come dei veri e propri "ammortizzatori". Questi meccanismi impediscono che anche pochi dati rumorosi alterino drasticamente il comportamento del modello, garantendo che i suoi risultati rimangano coerenti e affidabili anche quando la qualità del dataset degrada.

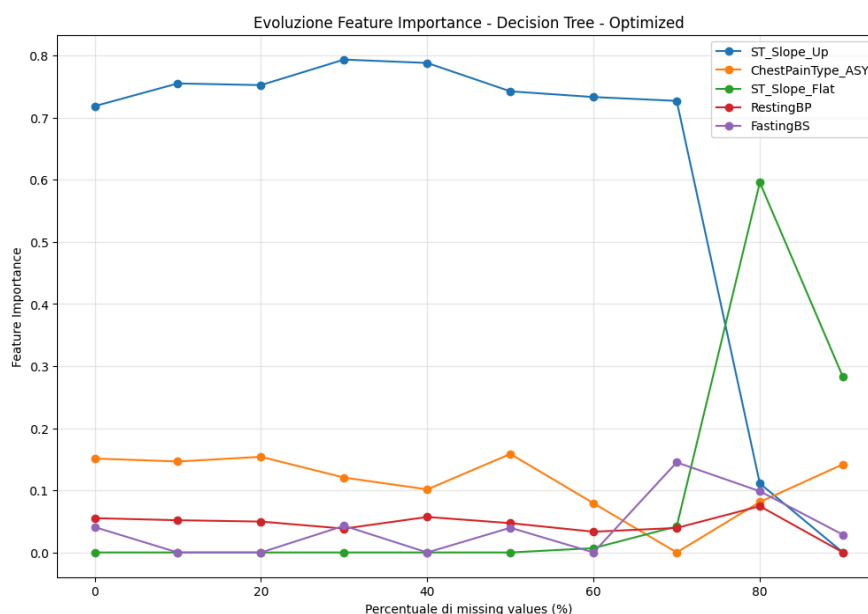
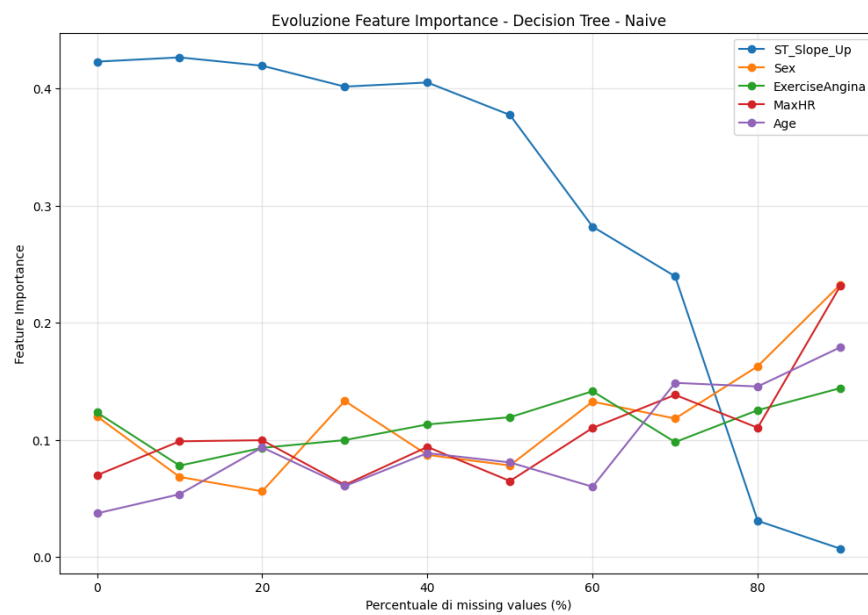
Oltre ad analizzare le performance delle metriche ho studiato anche l'evoluzione dei valori delle matrici di confusione al crescere della percentuale di valori mancanti.



I risultati mostrano come la Rete Neurale è risultata la più robusta: con imputazioni semplici come la mediana e la moda, mantiene prestazioni stabili dallo 0% al 90% di valori mancanti, senza grossi cali né oscillazioni. Il Decision Tree, invece, è più sensibile: nella versione “naive” con imputazione iterativa si nota un peggioramento oltre il 60% di valori mancanti, con aumento di errori e calo dei True Positives. La versione di Decision Tree ottimizzata registra performance molto più solide rispetto al suo corrispettivo naive. La SVM si è mostrata la più instabile: nel caso “naive” tende a collassare verso previsioni quasi tutte positive quando i

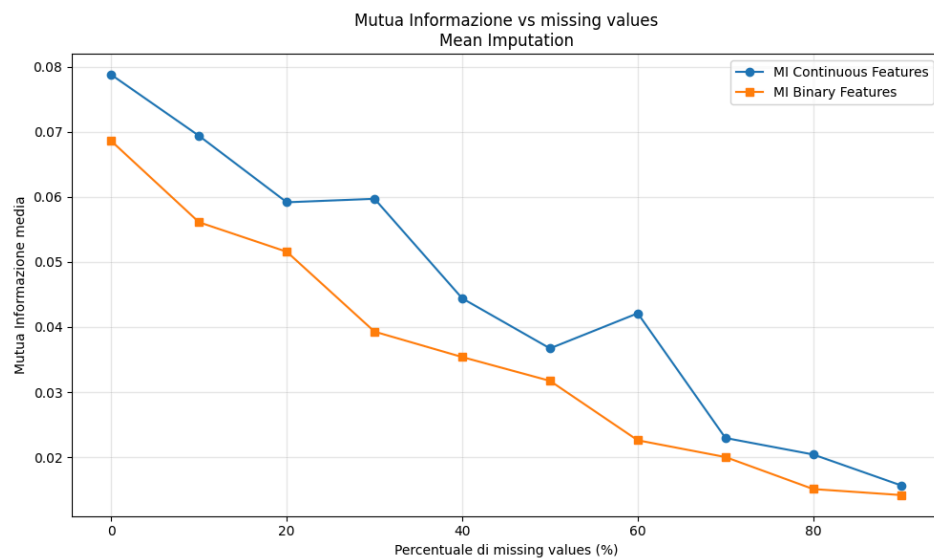
valori mancanti superano il 70%, mentre in versione ottimizzata ha un andamento molto irregolare, con forte peggioramento a percentuali estreme di dati mancanti.

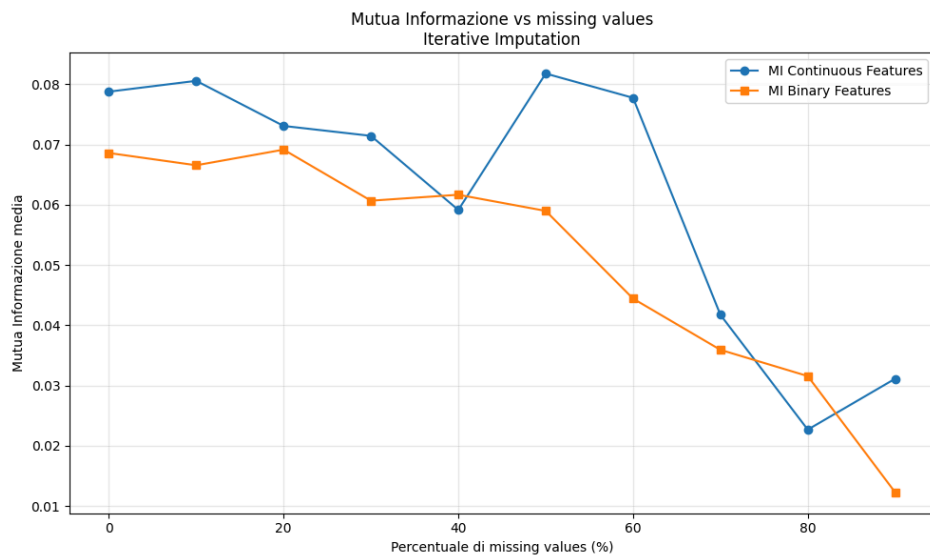
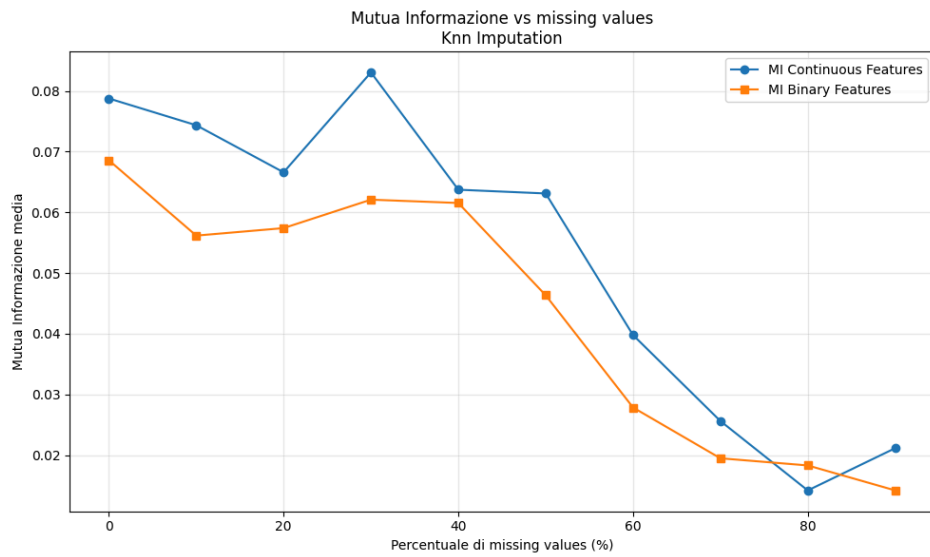
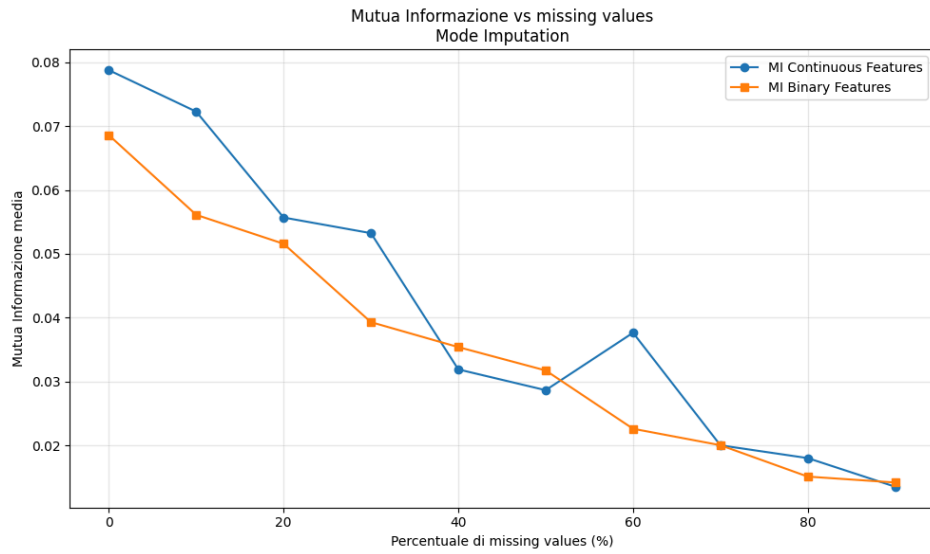
A questo punto sono andato a studiare anche l'evoluzione dell'importanza delle feature per comprendere meglio le cause dell'andamento delle metriche. Questa analisi è stato possibile svolgerla unicamente con gli alberi di decisione in quanto risulta computazionalmente molto meno oneroso. Infatti su SVM (con kernel non lineari) e reti neurali ho provato a studiare l'importanza delle metriche con shap ma per avere risultati un minimo significativi bisogna eseguirla con parametri non troppo piccoli il che comportava tempo di esecuzione di diverse ore rendendo l'analisi poca sostenibile



Dai grafici risulta che la feature più importante negli alberi di decisione era la ST_Slope_Up. Quando il modello fa crollare l'importanza di questa metrica a favore delle altre è il momento in cui si ha anche il calo di performance delle metriche più evidente sottolineando l'importanza di questa feature per questo modello.

Inoltre per osservare come lo sporcamento del dataset vada ad impattare sulla qualità delle informazioni che riesce a fornire, ho creato dei grafici che mostrano l'andamento della mutua informazione al crescere della degradazione del dataset. Avendo creato un grafico singolo per metodo di imputazione riusciamo anche ad analizzare quale tra questi metodi permette di mantenere più informazione.





Dai grafici è possibile osservare come la mutua informazione cala al crescere del livello di degradazione del dataset, come prevedibile. Inoltre dai grafici si può osservare che i metodi di imputazione che mantengono più informazione sono knn e l'iterativo che pagano però la loro maggiore solidità con tempi di esecuzione molto più alti rispetto agli altri metodi. Quindi in conclusione la scelta del miglior metodo di imputazione dipende dalle proprie esigenze. Se si ha una grande percentuale di valori mancanti e si cerca le migliori performance possibili allora knn e il metodo iterativo risultano essere le scelte migliori, mentre se si hanno pochi valori nulli o si ricerca maggiore efficienza allora uno tra media, mediana e moda risulta essere il metodo di imputazione migliore.

valori duplicati

I valori duplicati sono un problema estremamente comune nei dataset del mondo reale. Le cause della loro presenza possono essere diverse: un semplice errore umano durante l'inserimento, un sistema che registra più volte la stessa misurazione o, ancora più spesso, un'integrazione imperfetta di dati provenienti da fonti diverse che non riconoscono le informazioni già presenti. Studiare il comportamento dei modelli di machine learning quando si introducono dati duplicati è cruciale per capirne la **robustezza**. I modelli non sono infallibili: un eccesso di righe identiche può ingannare l'algoritmo, che potrebbe erroneamente pensare che quelle informazioni siano più importanti di altre. Questo fenomeno può portare all'**overfitting**, dove il modello impara a memoria i dati ripetuti invece di catturare i pattern generali.

implementazione

Per implementare progressivamente i valori duplicati ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare i duplicati è la seguente

```
def progressive_duplicate(df, steps=10, random_state=None):
    """
    Duplica progressivamente una percentuale crescente di righe del DataFrame.
    Restituisce una lista di DataFrame con duplicazioni crescenti.
    """
    np.random.seed(random_state)
    dfs = []
    n_rows = df.shape[0]
    for i in range(1, steps + 1):
        perc = i / steps
        n_dup = int(n_rows * perc)
        idx_to_duplicate = np.random.choice(df.index, n_dup, replace=False)
        df_dup = pd.concat([df, df.loc[idx_to_duplicate]], ignore_index=True)
        dfs.append(df_dup)
    return dfs

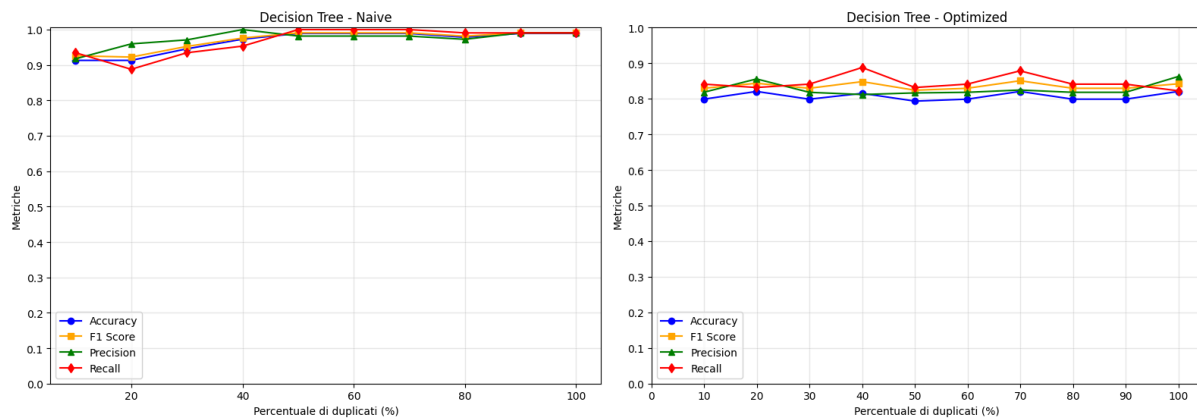
duplicated_dfs = progressive_duplicate(df, steps=10, random_state=42)
```

Questa funzione va a generare duplicati senza sovrascrivere i dati già presenti ma creandone di nuovi. In questo modo al termine del processo di degradazione otteniamo un dataset di dimensione doppia rispetto all'originale. Ho scelto questo approccio in quanto risulta essere quello più attinente alla realtà, infatti gli errori che portano a questa problematica vanno ad aggiungere nuove istanze al dataset senza sovrascrivere quelle già presenti (esempio: un'integrazione imperfetta di dati provenienti da fonti diverse che non riconoscono le informazioni già presenti).

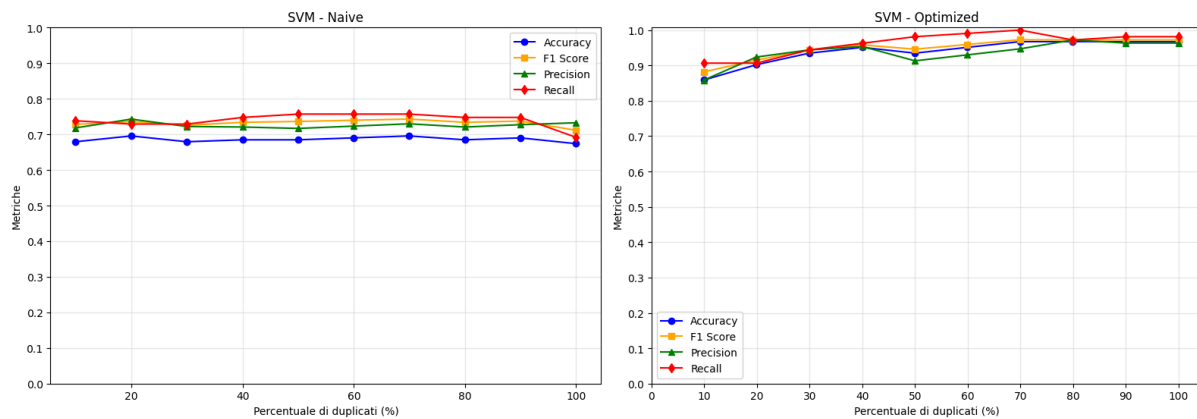
Analisi risultati

Per analizzare i risultati ho prodotto dei grafici che mostrano l'andamento delle metriche al crescere del livello di degradazione del dataset

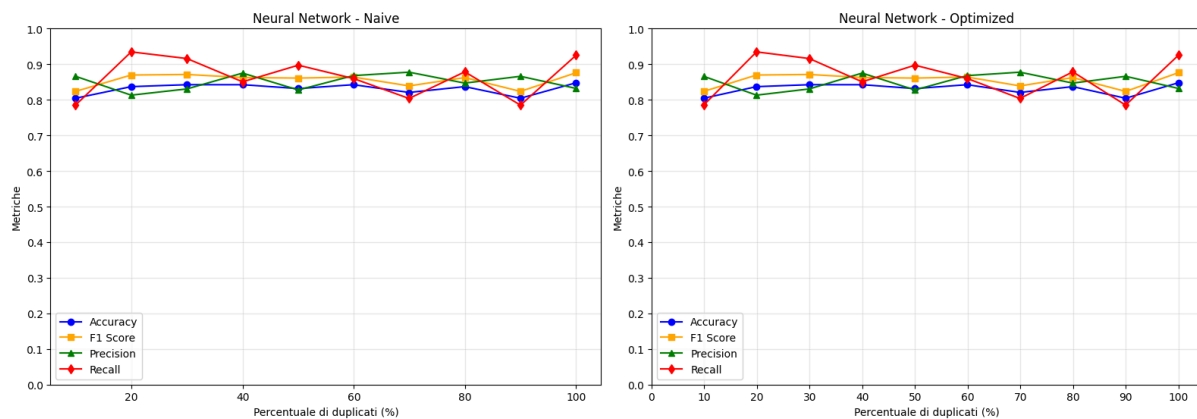
Confronto Decision Tree: Naive vs Optimized (duplicati)



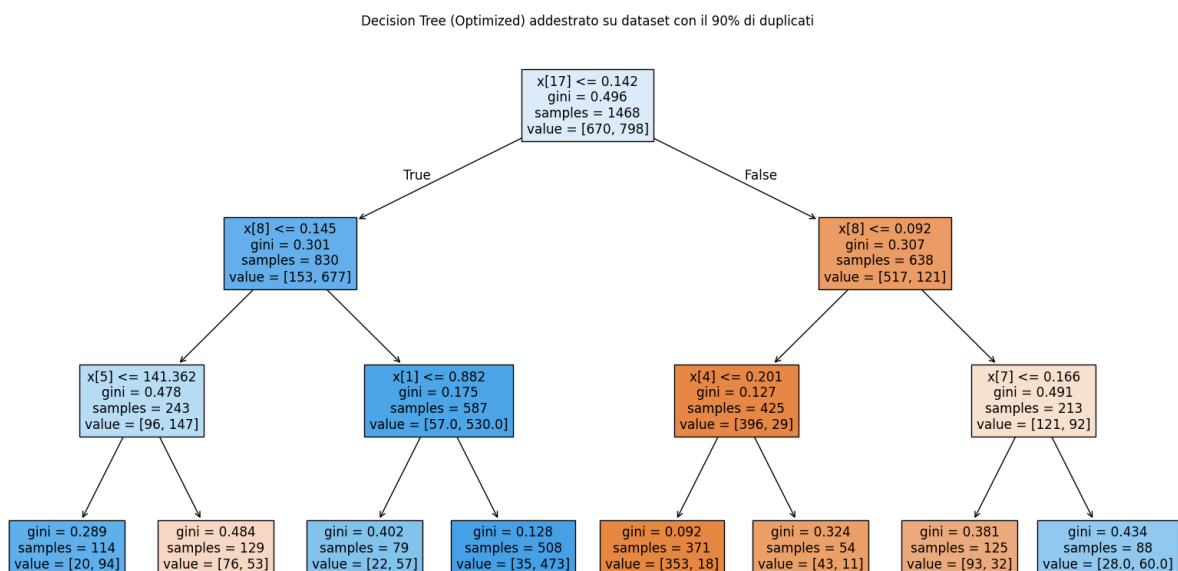
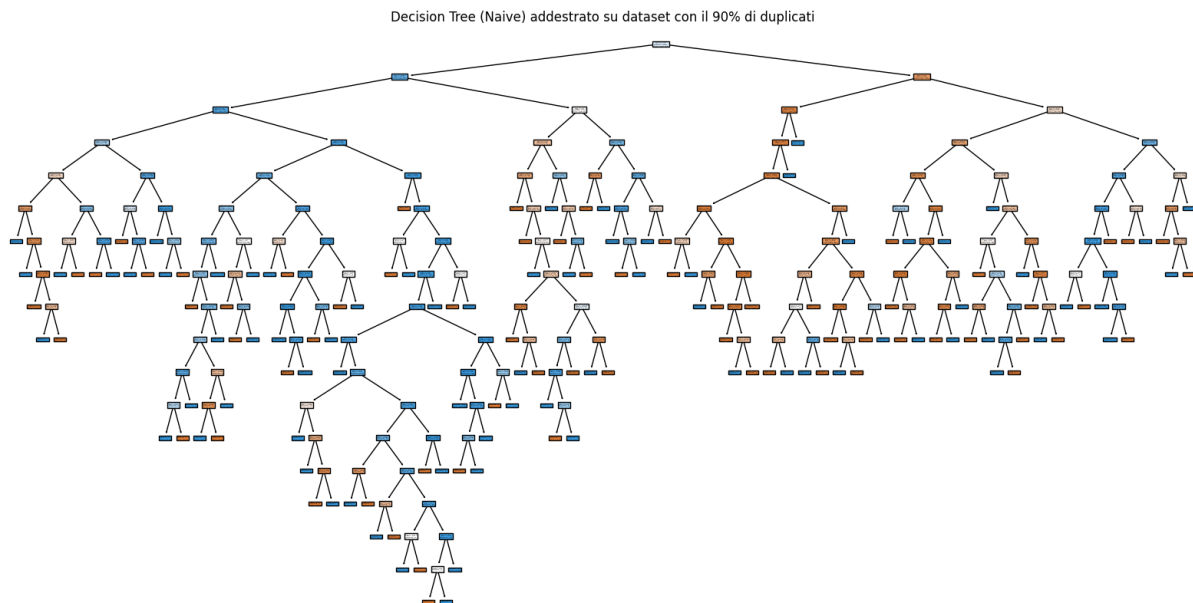
Confronto SVM: Naive vs Optimized (duplicati)



Confronto Neural Network: Naive vs Optimized (duplicati)



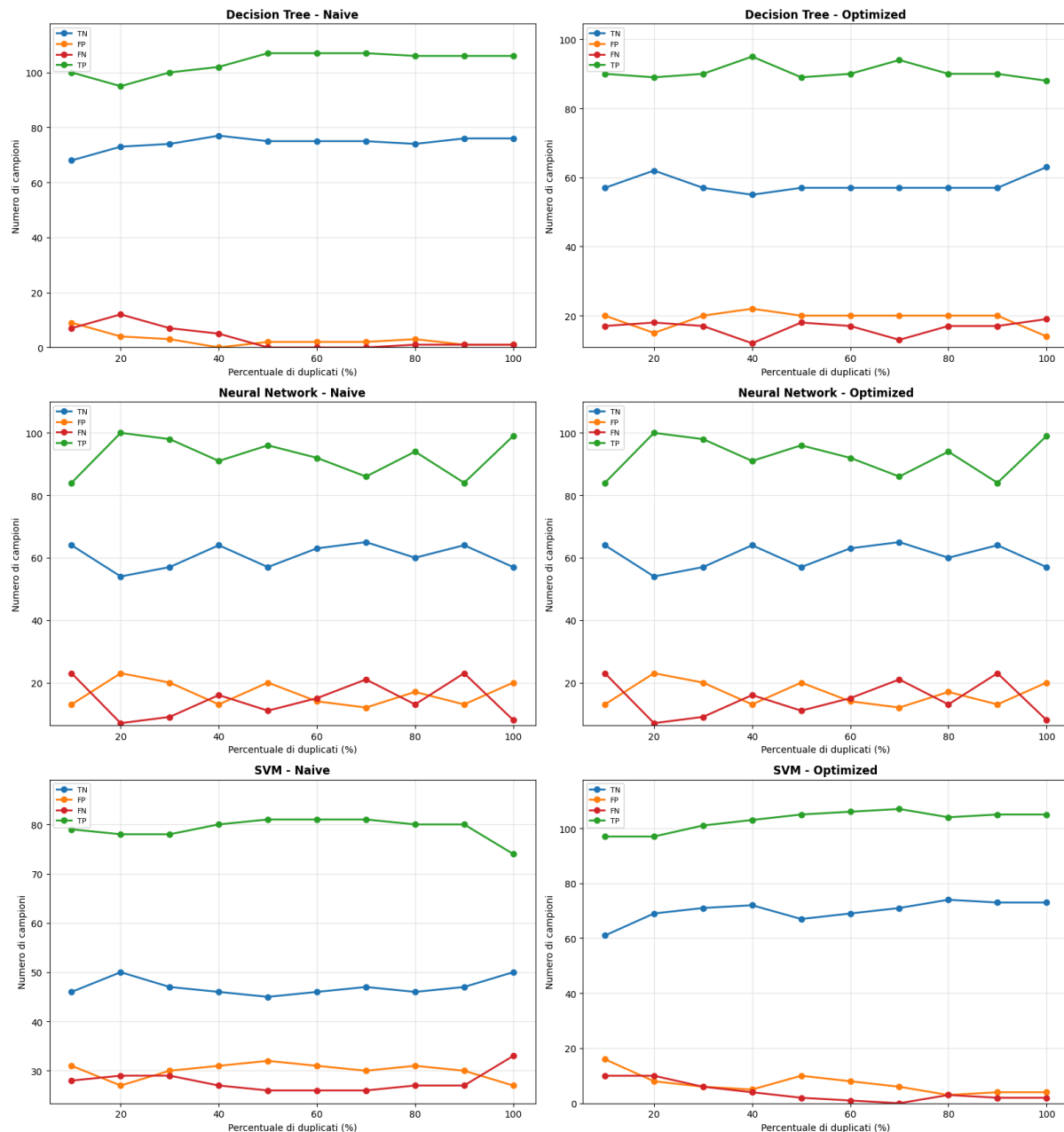
I grafici mostrano nel caso degli alberi di decisione un miglioramento delle performance, arrivando quasi a score perfetti delle metriche. Questo fenomeno è facilmente spiegabile con l'overfitting, infatti l'albero di decisione naive già con un dataset pulito soffre leggermente di overfitting come si può constatare dalla complessità dell'albero. Questo fenomeno non può fare altro che peggiorare se vado a duplicare tutte le righe, infatti il modello sarà ancora più fortemente basato unicamente sui dati di addestramento e farà sempre più fatica a generalizzare. Questa idea trova conferma nei risultati del grafico dell'albero di decisione ottimizzato. Infatti grazie al limite di profondità settato come parametro, l'albero di decisione è costretto a cercare di generalizzare e non può adattarsi troppo ai dati di training.



Per la SVM naive, le metriche rimangono praticamente costanti con la degradazione del dataset. La SVM ottimizzata, invece, riesce a sfruttare i duplicati: i pattern diventano più stabili, il margine più solido e le performance migliorano molto, fino a valori altissimi, soprattutto sul recall. Oltre un certo livello però il guadagno si satura, e si vede anche una piccola flessione

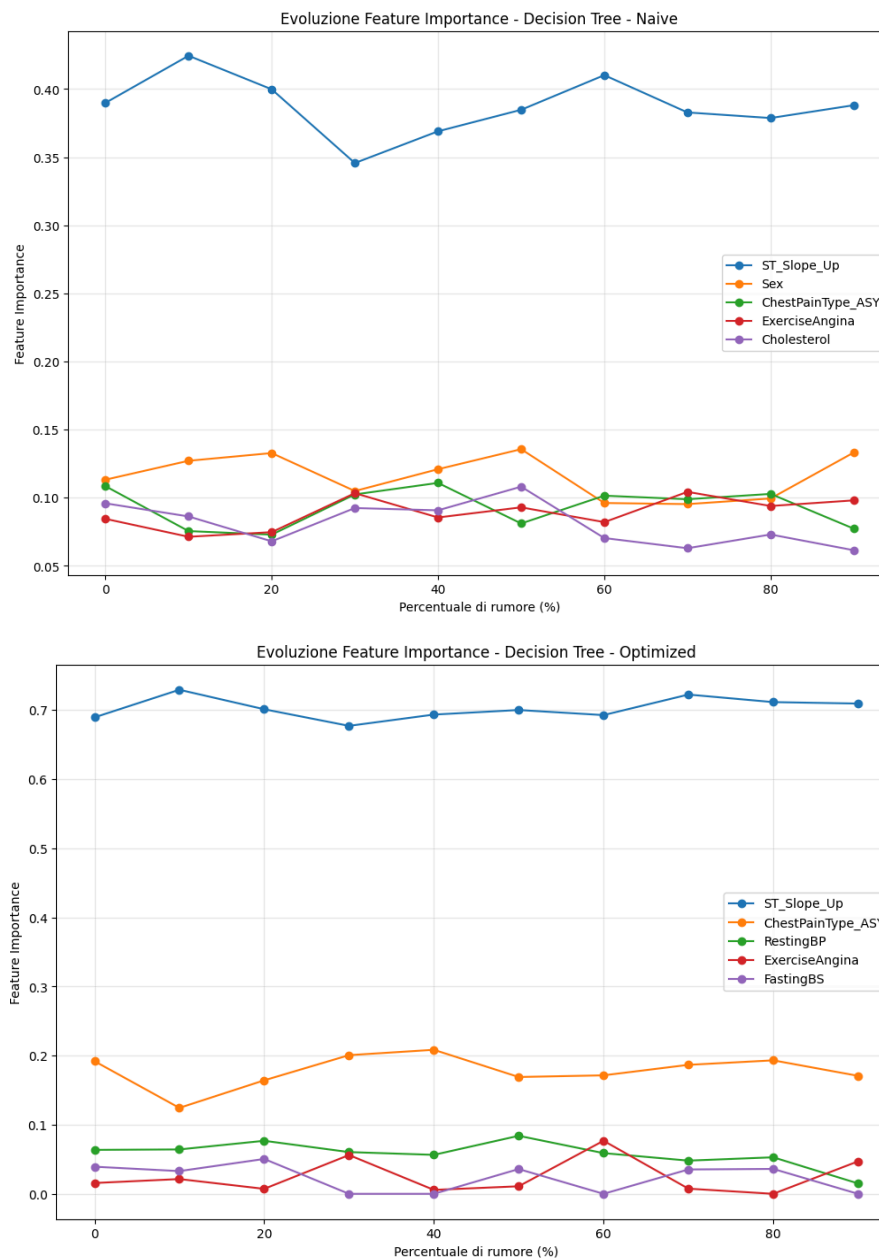
perché troppa ridondanza non aggiunge informazione e può perfino introdurre un leggero overfitting. Per quanto riguarda invece le reti neurali si ha un comportamento molto stabile del modello sia per quanto riguarda la versione naive che quella ottimizzata, che non riscontrano grosse variazioni di performance rispetto a quelle registrate con il dataset pulito. Le reti neurali non beneficiano particolarmente dei duplicati perché già normalmente vedono lo stesso dato molte volte in fase di training e quindi i duplicati hanno impatto molto meno rispetto a modelli come SVM in cui invece i duplicati vanno ad alterare la geometria dello spazio.

Di seguito ho riportato i grafici che mostrano l'andamento delle matrici di confusione al crescere del numero di duplicati



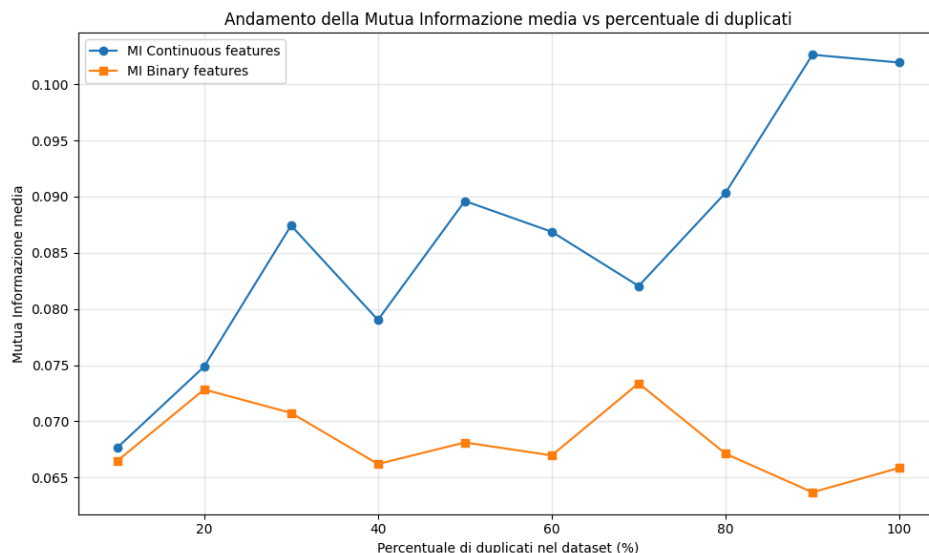
Da questi grafici si può osservare come i duplicati non abbiano causato grossi danni a livello di falsi positivi e falsi negativi, infatti questi seppur con andamenti irregolari, rimangono nel complesso piuttosto bassi.

Oltre ad analizzare le performance delle metriche sono andato a studiare anche l'evoluzione dell'importanza delle feature negli alberi di decisione, per comprendere meglio le cause dell'andamento delle metriche



Dai grafici risulta ancora che la feature più importante negli alberi di decisione è la ST_Slope_Up. Il degradamento del dataset con i duplicati non va ad intaccare la scelta delle feature, che rimane pressoché costante per tutti gli step di "sporciamento".

Inoltre per osservare come lo sporciamento del dataset vada ad impattare sulla sua qualità nel dare informazioni significative ho creato dei grafici che mostrano l'andamento della mutua informazione al crescere della degradazione del dataset.



La mutua informazione delle feature continue cresce in maniera direttamente proporzionale al numero di duplicati aggiunti, seppur con un andamento irregolare. Invece la mutua informazione delle feature binarie ha un andamento irregolare in cui però non si registrano particolari miglioramenti per quanto riguarda la mutua informazione. La differenza osservata può essere spiegata considerando la natura delle due tipologie di variabili. Le **feature continue**, avendo un range di valori molto ampio, quando vengono duplicate rafforzano le relazioni già presenti con la variabile target: la ripetizione di determinati valori numerici aumenta la probabilità congiunta di specifiche coppie (x,y) , rendendo più evidente la dipendenza statistica. Questo si traduce in un incremento progressivo e regolare della mutua informazione al crescere dei duplicati. Le **feature binarie**, invece, possono assumere solo due stati (0 o 1). La duplicazione di queste osservazioni non modifica le frequenze relative delle due categorie e, di conseguenza, non aggiunge nuova informazione rispetto al target. Per questo motivo la mutua informazione rimane sostanzialmente stabile, con eventuali piccole oscillazioni dovute a effetti di stima o di campionamento, ma senza veri miglioramenti in termini di capacità predittiva.

outliers

Gli outliers sono valori che si discostano notevolmente dagli altri dati, e la loro presenza può influenzare negativamente l'analisi e la modellazione. Le ragioni della loro comparsa sono molteplici:

- **Errori di misurazione:** Un sensore malfunzionante o un errore di immissione dati può registrare un valore completamente fuori scala.
- **Variazioni naturali ma rare:** Alcuni outliers possono essere dati legittimi ma estremamente rari, come una transazione finanziaria insolitamente grande o un evento meteorologico estremo.

Analizzare il comportamento di un modello all'aumentare del numero di outliers fornisce una comprensione profonda della sua robustezza e del suo comportamento in condizioni reali.

Implementazione

Per implementare progressivamente gli outliers ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare gli outliers è la seguente:

```
def add_continuous_outliers(self, contamination_rate=0.02, methods=['extreme'],
seed=None):
    """Contamina singoli valori numerici"""
    rng = np.random.default_rng(seed)
    if not self.continuous_features:
        return {}

    univariate_methods = [m for m in methods if m != 'multivariate']
    if not univariate_methods:
        univariate_methods = ['extreme']

    # Calcola quanti nuovi valori contaminare
    n_new_outliers = int(round(self.total_numeric_values * contamination_rate))

    if n_new_outliers == 0 and contamination_rate > 0:
        n_new_outliers = 1

    # Genera tutte le possibili combinazioni (row_label, feature)
    all_value_positions = []
    for row_label in self.current_df.index:
        for feature in self.continuous_features:
            all_value_positions.append((row_label, feature))

    # Filtra quelle già contaminate
    available_positions = [pos for pos in all_value_positions
                           if pos not in self.contaminated_values]

    if len(available_positions) == 0:
        print("No more clean values available for contamination.")
        return {}

    if len(available_positions) < n_new_outliers:
        n_new_outliers = len(available_positions)
        print(f"Warning: Only {len(available_positions)} clean values remaining. Using all of them.")

    # Scegli casualmente le posizioni da contaminare
    chosen_positions = rng.choice(len(available_positions), size=n_new_outliers,
replace=False)
    chosen_positions = [available_positions[i] for i in chosen_positions]

    contamination_info = {}

    # Assicurati di poter ripristinare i valori originali
    if not hasattr(self, 'original_values'):
        self.original_values = {}

    for row_label, feature in chosen_positions:
        method = str(rng.choice(univariate_methods))

        # Salva il valore originale (prima volta)
```

```

        if row_label not in self.original_values:
            self.original_values[row_label] = {}
        if feature not in self.original_values[row_label]:
            self.original_values[row_label][feature] = self.current_df.at[row_label,
feature]

        # Dati puliti: escludi valori già contaminati per questa feature
        clean_positions = [(r1, f) for r1, f in all_value_positions
                           if f == feature and (r1, f) not in self.contaminated_values]
        clean_data = pd.Series([self.current_df.at[r1, f] for r1, f in
clean_positions]).dropna()

        if clean_data.empty:
            continue

        q1, q3 = clean_data.quantile(0.25), clean_data.quantile(0.75)
        iqr = q3 - q1
        std_val = float(clean_data.std()) if clean_data.std() != 0 else 1.0
        min_val, max_val = clean_data.min(), clean_data.max()

        orig_val = float(self.current_df.at[row_label, feature])

        # Genera l'outlier
        if method == 'extreme':
            if rng.random() < 0.5:
                outlier_val = q1 - rng.uniform(3, 6) * iqr
                outlier_val = max(outlier_val, float(min_val) - 3 * std_val)
            else:
                outlier_val = q3 + rng.uniform(3, 6) * iqr
                outlier_val = min(outlier_val, float(max_val) + 3 * std_val)
        elif method == 'shift':
            shift_factor = rng.choice([-1, 1]) * rng.uniform(2, 4)
            outlier_val = orig_val + shift_factor * std_val
        elif method == 'noise':
            noise_factor = rng.choice([-1, 1]) * rng.uniform(3, 5)
            outlier_val = orig_val + noise_factor * std_val
        else:
            # fallback
            outlier_val = q3 + rng.uniform(3, 6) * iqr

        # Domain-aware clipping
        if feature in ['Age', 'RestingBP', 'Cholesterol', 'MaxHR']:
            lower_bound = 0
            outlier_val = max(outlier_val, lower_bound)

        # Preserva il tipo intero se necessario
        if pd.api.types.is_integer_dtype(self.current_df[feature].dtype):
            outlier_val = int(round(outlier_val))

        # Applica la modifica
        self.current_df.at[row_label, feature] = outlier_val

        # Registra l'informazione
        contamination_info.setdefault(feature, []).append({
            'index': row_label,
            'original_value': self.original_values[row_label][feature],

```

```

        'outlier_value': outlier_val,
        'method': method
    })

    # Aggiorna il set dei valori contaminati
    self.contaminated_values.update(set(chosen_positions))

    return contamination_info

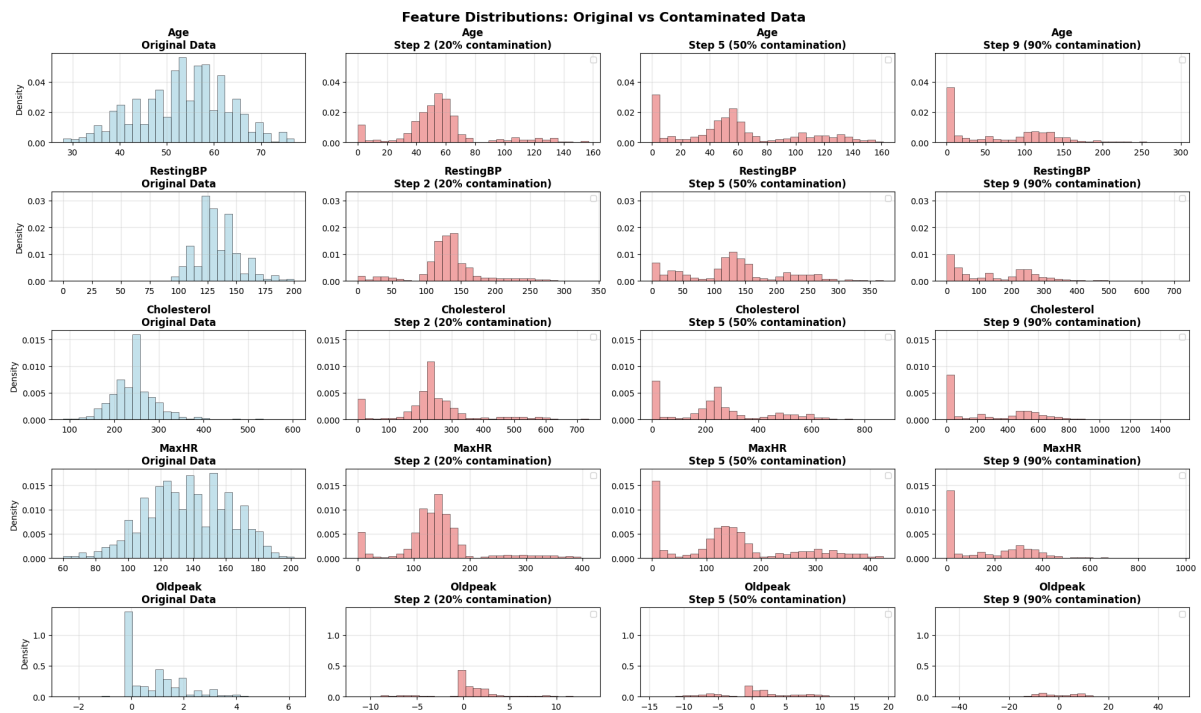
```

Questa funzione genera outliers per le feature numeriche presenti nel dataset. Vengono generati 3 diversi tipi di outliers:

- **extreme**: produce valori che si discostano molto dalla distribuzione iniziale e sono utili per simulare gli errori più evidenti come malfunzionamenti di sensori o errore di inserimento dati. Questi outliers vengono generati utilizzando il metodo del range interquartile (IQR). Questa tecnica statistica per identificare valori anomali si sviluppa nel seguente modo:
 - a. Si calcolano i quartili
 - b. Si calcola l'interquartile (IQR) ovvero la differenza tra il terzo e il primo quartile. Questo valore rappresenta la "diffusione" dei dati centrali
 - c. Si calcolano i limiti oltre i quali i valori vengono considerati outliers. Limite inferiore: $Q1 - 1.5 \cdot IQR$. Limite superiore: $Q3 + 1.5 \cdot IQR$
 - d. Vengono generati valori casuali che si trovano al di fuori di questi confini
- **shift**: questo metodo sposta il valore originale di un multiplo della deviazione standard. Risulta utile in quanto simula scenari come l'errore di conversione dell'unità di misura
- **noise**: Questo metodo aggiunge un rumore casuale ma significativo al valore originale. L'entità del rumore è sempre un multiplo della deviazione standard. Questo metodo è particolarmente utile per simulare anomalie che si verificano naturalmente, dove i dati non sono intrinsecamente "sbagliati", ma sono influenzati da una forte variabilità o da misurazioni imprecise. Questo è comune in dati finanziari o medici. Studiare questo tipo di outlier aiuta a testare la resilienza dei modelli a un rumore di fondo più realistico.

Analisi risultati

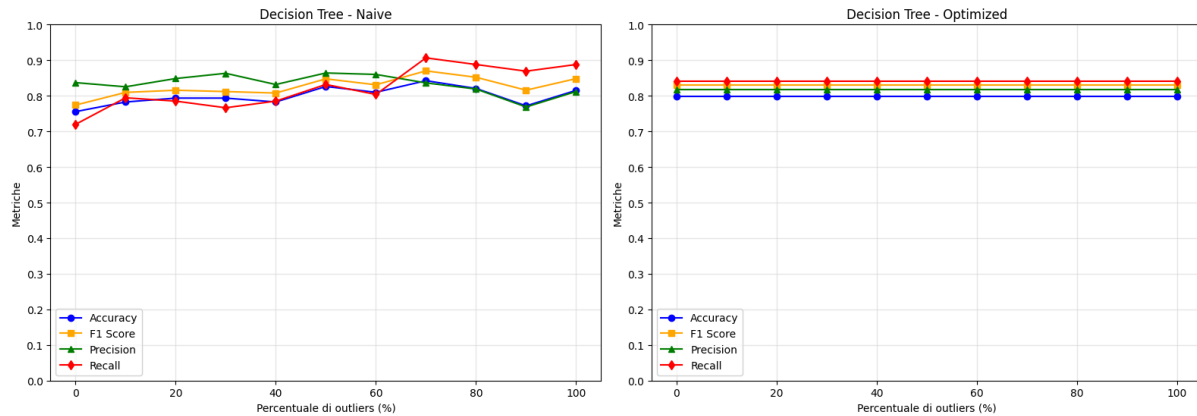
Per prima cosa ho creato dei grafici per visualizzare gli outliers inseriti e come andassero ad impattare sulla distribuzione dei dati iniziale



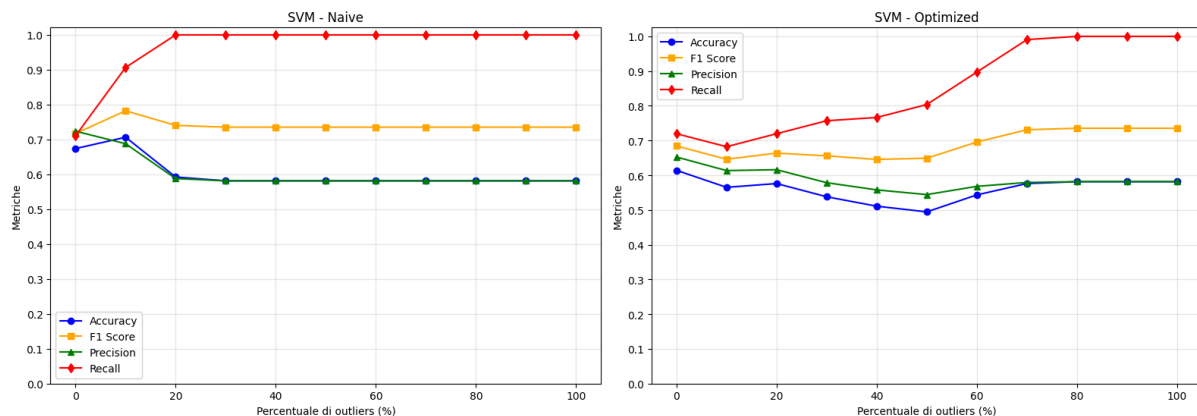
Dai grafici risulta che l'inserimento di outliers, come prevedibile, ha portato ad una forte modifica della distribuzione iniziale passando da distribuzioni normali a distribuzioni molto più appiattite e disperse. Osservando l'evoluzione delle distribuzioni delle feature numeriche ai diversi step di contaminazione emerge che in molte variabili gli outlier tendono a concentrarsi verso i valori inferiori, spesso in prossimità dello zero. Questo fenomeno può essere spiegato considerando sia la natura del dataset originale, sia dai vincoli applicati durante la generazione degli outlier. Le variabili analizzate, come ad esempio Age, RestingBP, Cholesterol e MaxHR, presentano valori strettamente positivi e distribuiti in intervalli relativamente ridotti. Quando il metodo di contaminazione produce valori estremi sottraendo multipli dell'intervallo interquartile dal primo quartile, il risultato porta naturalmente alla generazione di valori inferiori a zero. Tuttavia, il codice di contaminazione impone un vincolo che impedisce a queste feature di assumere valori negativi, bloccandole al valore minimo consentito pari a zero. La combinazione di queste due condizioni, ossia la tendenza della procedura a spingere alcuni valori verso il basso e la correzione automatica che impedisce di superare il limite inferiore, determina un accumulo artificiale di outlier intorno a zero. In questo modo si spiegano i picchi evidenziati nei grafici in corrispondenza dei valori più bassi delle distribuzioni contaminate.

Successivamente sono andato a creare i grafici per mostrare l'andamento delle metriche al crescere del numero di outliers

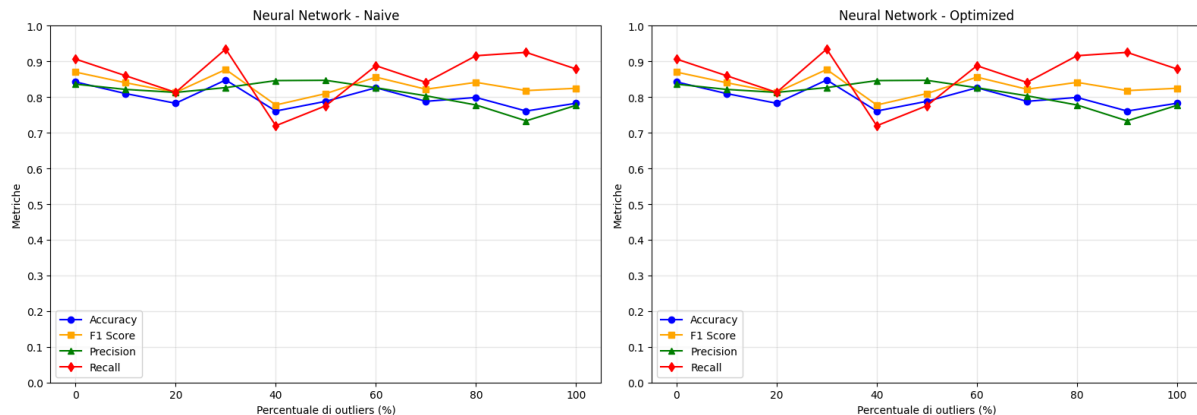
Confronto Decision Tree: Naive vs Optimized



Confronto SVM: Naive vs Optimized



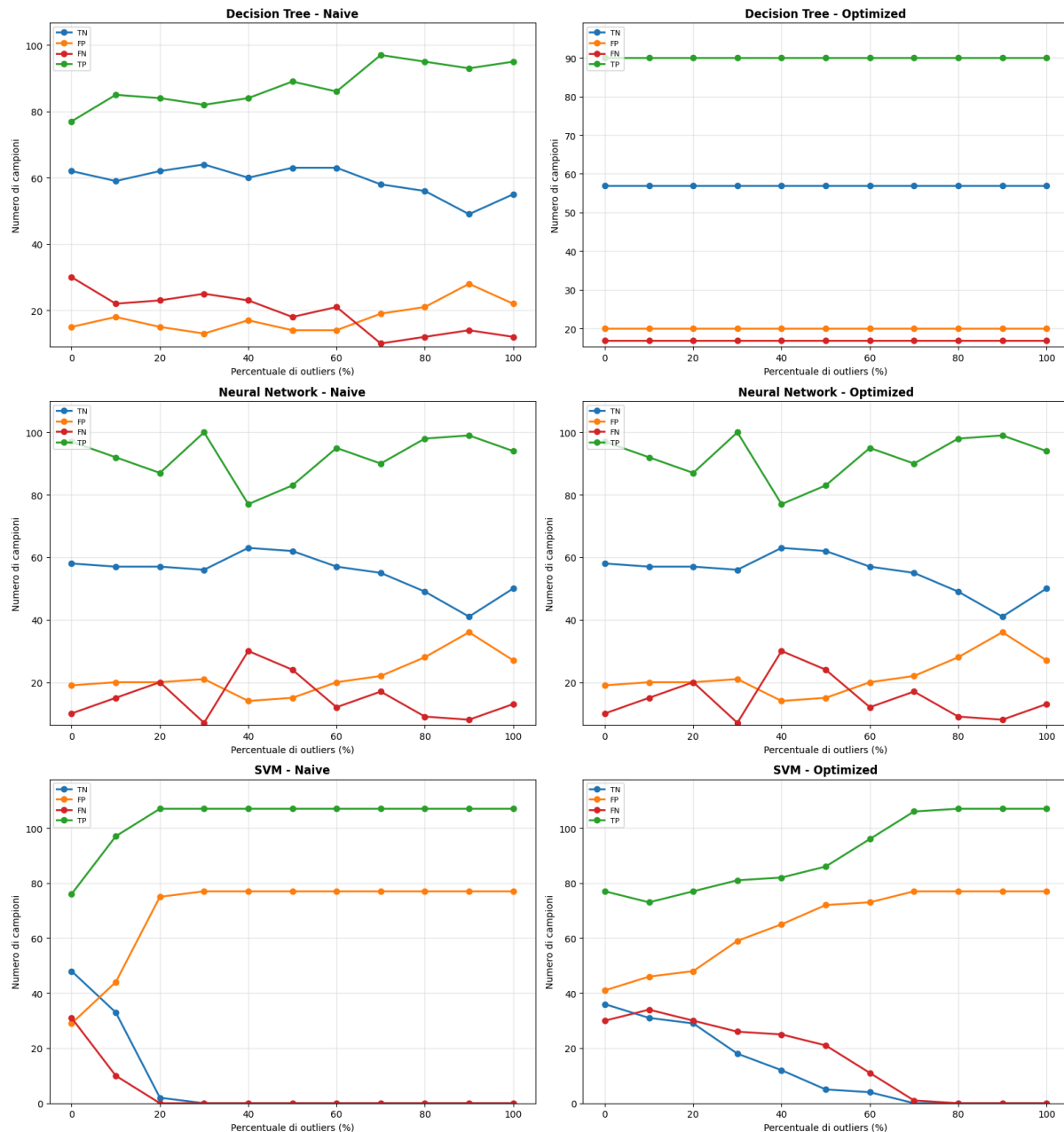
Confronto Neural Network: Naive vs Optimized



Dai grafici si evince che le performance migliori e più stabile sono state registrate dagli alberi di decisione e delle reti neurali. In particolare si osserva come l'albero di decisione ottimizzato ha registrato performance identiche al dataset pulito. Questo comportamento può essere spiegato dal fatto che l'albero ottimizzato utilizzi unicamente feature binarie per fare previsione e non usi nessuna feature continue. Essendo che le uniche feature degrade con gli outliers sono state quelle continue questo risultato appare completamente giustificato. Invece svm soffre particolarmente gli outliers con performance mediocri che arrivano a toccare valori sotto 0.6 di accuracy (poco più di una scelta casuale) con outliers pari al 20% del dataset. Dalla recall altissima abbinata ad un accuracy mediocre si può constatare come il modello tenda a classificare tutte le istanze come positive (questa ipotesi verrà rafforzata dai

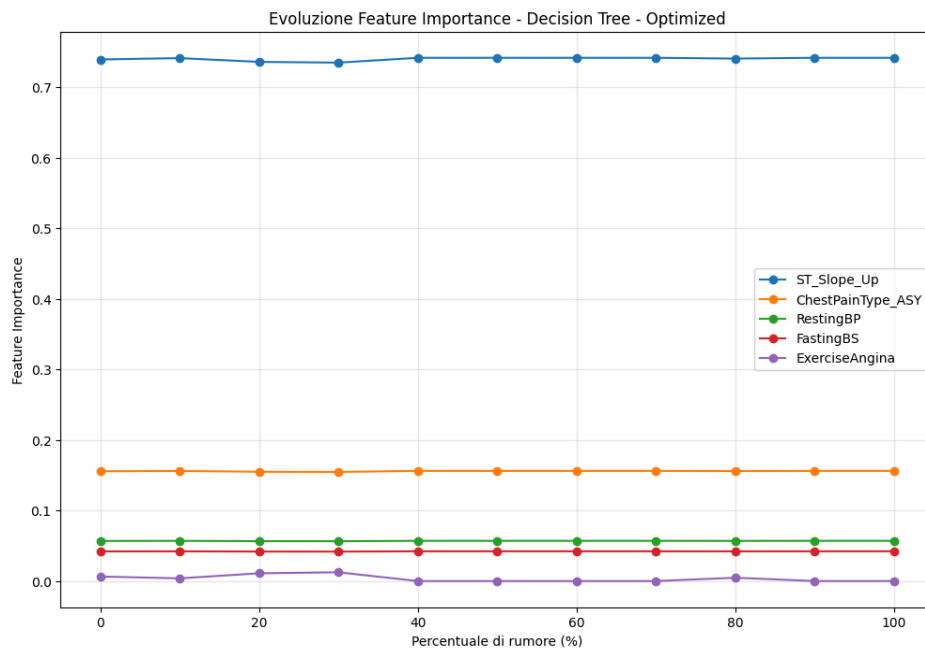
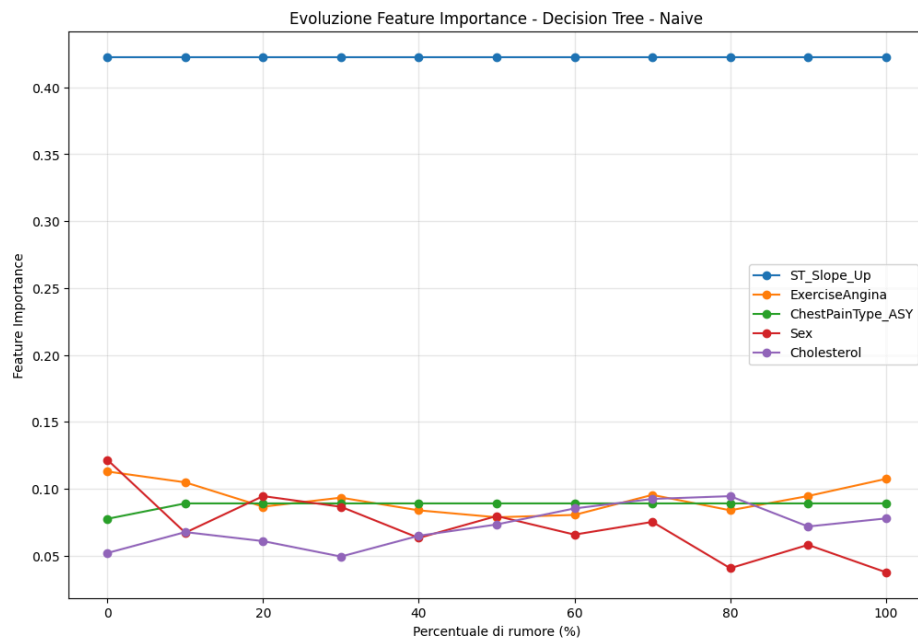
grafici successivi sull'andamento delle matrici di confusione). Inoltre si può ancora una volta constatare come i modelli ottimizzati hanno registrato performance molto più stabili rispetto ai modelli naive.

Di seguito sono presenti i grafici contenenti l'evoluzione delle matrici di confusione al crescere del numero di outliers



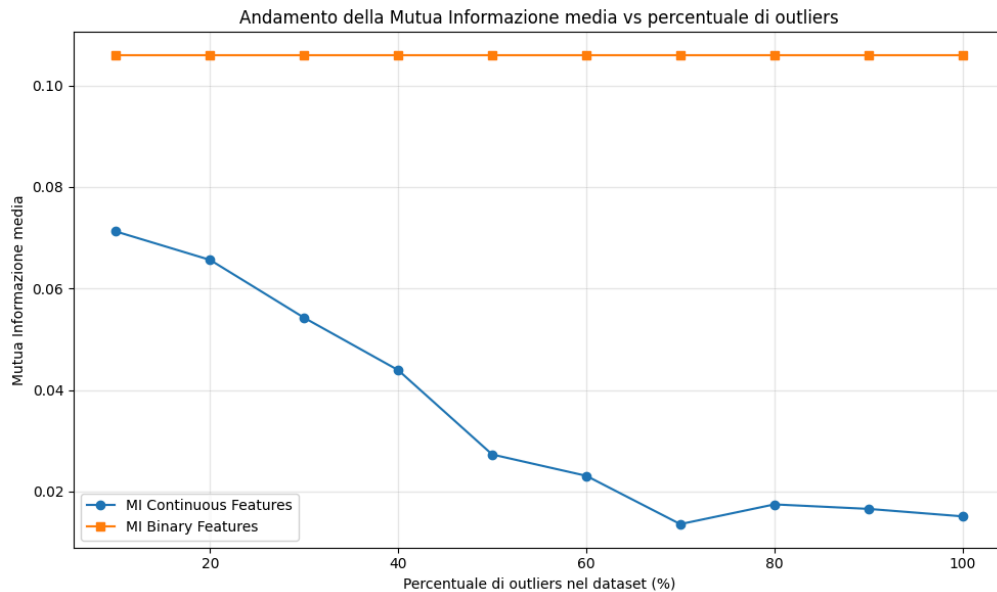
Dai grafici di svm si può osservare un numero molto alto di true positive e false positive. Questo trend va a confermare le ipotesi fatte in precedenza ovvero come i modelli svm in questo caso tendono a classificare tutte le istanze come positive. Invece anche da questi grafici decision tree e reti neurali mostrano performance piuttosto solide.

I seguenti grafici riportano l'evoluzione dell'importanza delle feature al crescere del numero di outliers inseriti nel dataset



Questi grafici confermano quanto detto in precedenza, infatti si può osservare come l'importanza delle feature nell'albero ottimizzato rimangano praticamente sempre costanti. Questo è dovuto al fatto che utilizzi unicamente feature binarie che non sono intaccate dagli outliers. Invece nell'albero naive si possono osservare variazioni nell'importanza delle feature a causa della presenza di feature numeriche come Cholesterol.

Infine riporto anche i grafici sulla variazione della mutua informazione al crescere del numero di outliers



Da questo grafico si osserva come la qualità delle feature numeriche vada a calare al crescere del numero di outliers. Invece la mutua informazione delle feature binarie rimane costante in quanto non vengono modificate dal processo di inserimento di outliers.

rumore su feature target

Il rumore della variabile target è un problema comune. Può essere causato da errori umani nel data entry, diagnosi mediche sbagliate, etichettature imprecise da parte di sistemi automatici o annotazioni scorrette in grandi dataset. Studiare l'impatto di questo rumore è utile per cercare di capire quali tipi di modelli sono più solidi rispetto a questo tipo di problematica.

implementazione

Per implementare progressivamente il rumore sulla variabile target ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare il rumore è la seguente

```
def add_progressive_noise(df: pd.DataFrame, target_column: str) -> dict:
    # Controlla che la colonna target sia binaria (contenente solo 0 e 1)
    if not all(df[target_column].isin([0, 1])):
        raise ValueError(f"La colonna '{target_column}' non è binaria (deve contenere solo 0 e 1).")

    noisy_datasets = {}
    n_samples = len(df)

    print(f"DataFrame originale con {n_samples} righe.")

    # Itera attraverso le percentuali di rumore da 0 a 100 con step di 10
    for percentage in range(0, 101, 10):
        # Lavoriamo sempre su una copia pulita del df originale per evitare rumore cumulativo
        df_noisy = df.copy()

        # Se la percentuale è 0, non facciamo nulla e salviamo la copia originale
```

```

if percentage == 0:
    noisy_datasets[f'0%_noise'] = df_noisy
    continue

# Calcola il numero di etichette da "flippare" (invertire)
n_to_flip = int(np.floor(n_samples * (percentage / 100)))

# Seleziona casualmente gli indici delle righe da modificare
# Usiamo random_state per la riproducibilità degli esperimenti
indices_to_flip = df.sample(n=n_to_flip, random_state=42).index

# Inverti i valori nella colonna target per gli indici selezionati
df_noisy.loc[indices_to_flip, target_column] = ~df_noisy.loc[indices_to_flip,
target_column]

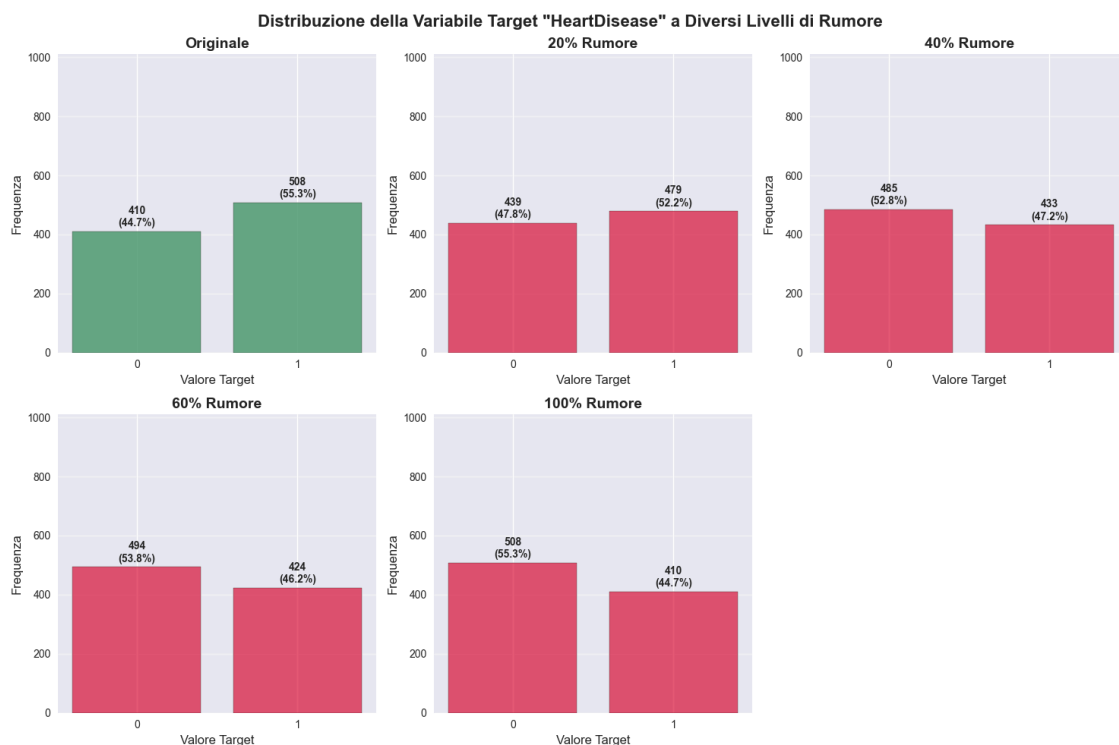
# Salva il DataFrame rumoroso nel dizionario
key_name = f'{percentage}%_noise'
noisy_datasets[key_name] = df_noisy
print(f"Generato dataset con {percentage}% di rumore ({n_to_flip} etichette
invertite).")

return noisy_datasets

```

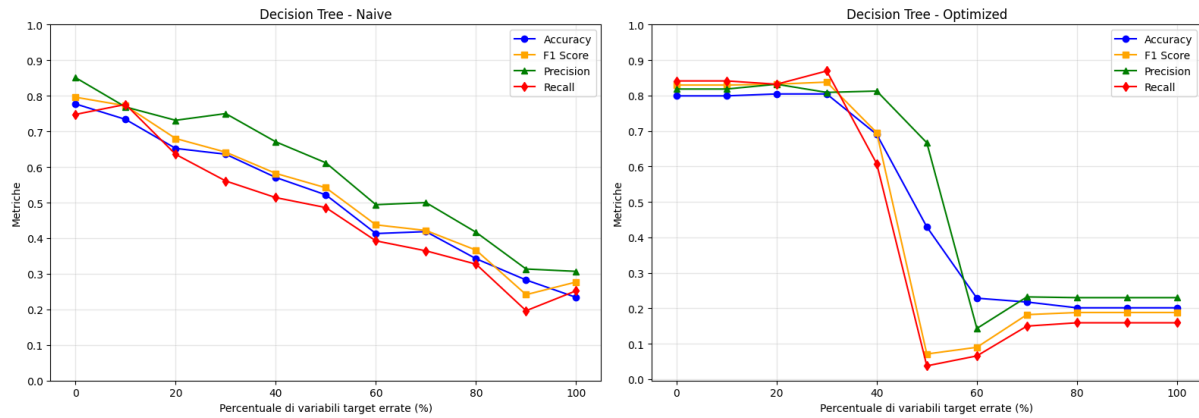
Analisi risultati

Come prima cosa sono andato a visualizzare come evolve la distribuzione della feature target al crescere del rumore.

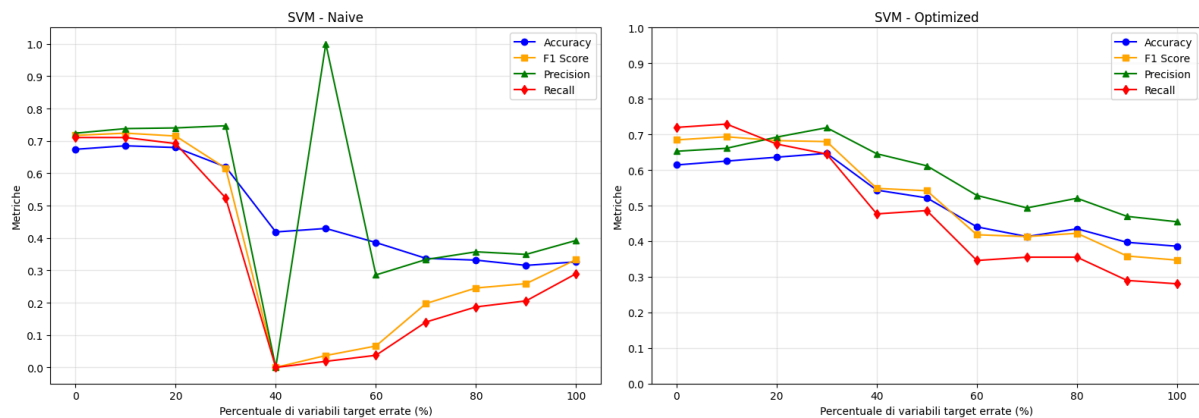


Subito dopo ho creato i grafici che mostrano l'andamento delle metriche al crescere del numero di outliers.

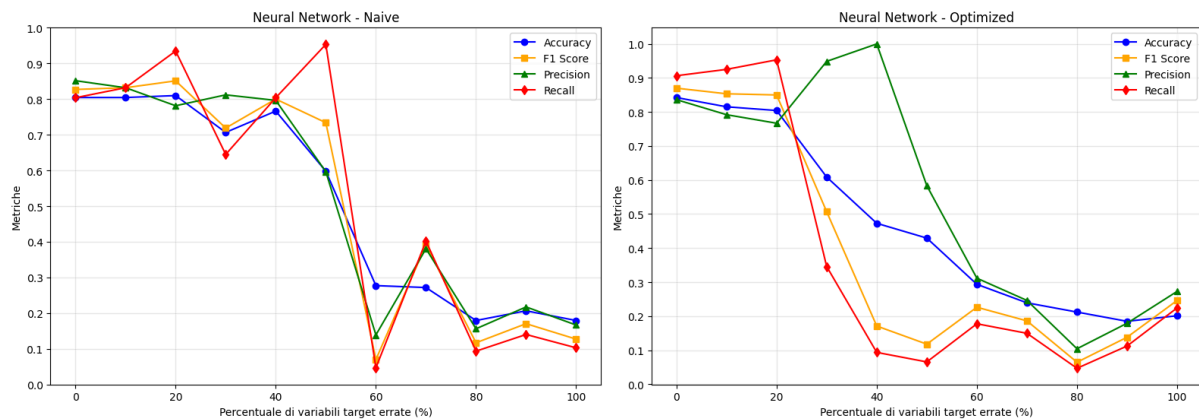
Confronto Decision Tree: Naive vs Optimized (duplicati)



Confronto SVM: Naive vs Optimized (duplicati)



Confronto Neural Network: Naive vs Optimized (duplicati)



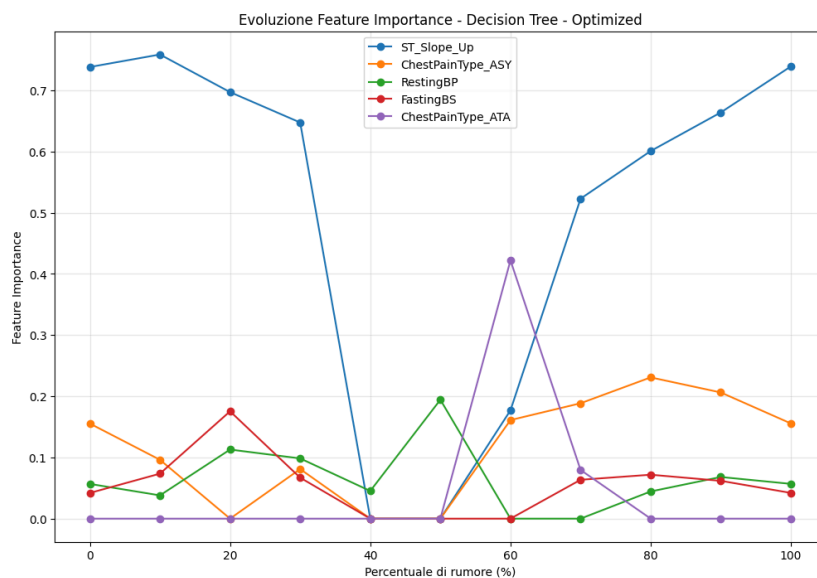
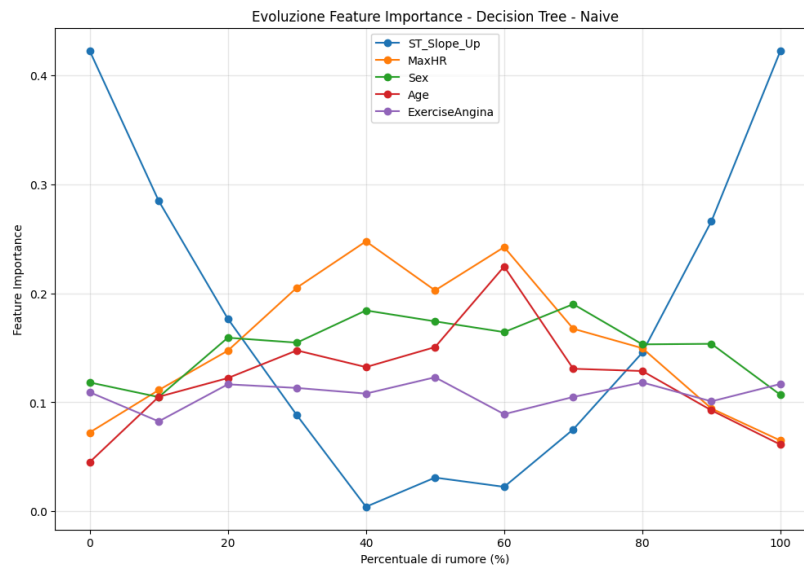
I risultati mostrano un degrado progressivo delle performance per tutti i modelli all'aumentare della percentuale di corruzione della feature target. L'albero decisionale naive e la SVM ottimizzata presentano un decadimento più lineare, mentre l'albero ottimizzato mostra un andamento non monotono con un picco di errore intorno al 50% di corruzione. L'apparente leggero miglioramento delle performance oltre il 50% di corruzione può essere spiegato considerando che, con livelli di corruzione molto elevati, il modello impara efficacemente la relazione invertita tra features e target alterato. Tuttavia, è importante notare che questa 'miglior performance' si riferisce alla predizione del target corrotto, non di quello originale.

Di seguito invece riporto l'andamento delle matrici di confusione al crescere del livello di degradazione delle feature target.



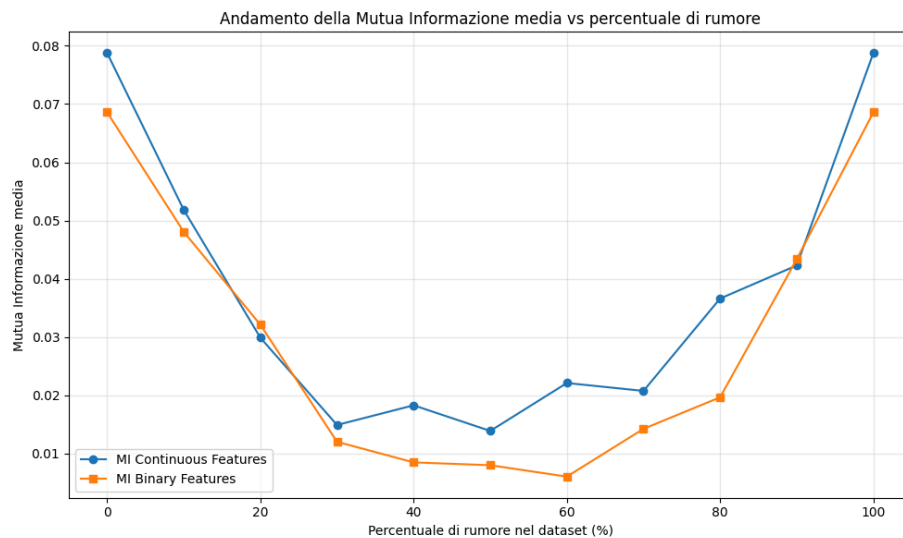
Dai grafici si può osservare, anche se con andamento irregolare, un trend progressivo di decrescita dei True Positive e True Negative al crescere della percentuale di rumore. Al contrario si registra una crescita progressiva dei Falsi Positivi e Falsi negativi al crescere della percentuale di rumore. Questo trend rispecchia in pieno le aspettative teoriche.

Di seguito ho invece riportato l'evoluzione dell'importanza delle feature negli alberi di decisione



Dai grafici risulta come la feature più importante ST_Slope_Up crolla di importanza intorno al 50% del livello di contaminazione che nell'albero di decisione ottimizzato corrisponde anche al livello con le performance più basse registrate. Infatti appena si torna ad utilizzare ST_Slope_Up si ha un trend di crescita delle performance confermando l'importanza di questa feature per questo modello. Lo stesso trend si ha anche nell'albero di decisione naive, dove però non si ha un picco negativo di performance intorno al 50% ma invece si ha una decrescita graduale. Questo può essere motivato da una scelta migliore delle feature sostitutive ad ST_Slope_Up.

Infine riporto anche i grafici sulla mutua informazione registrata al crescere del livello di degradazione della feature target.



Il grafico della mutua Informazione media mostra un andamento a U, con valori minimi intorno al 50% di rumore. Questo conferma che la relazione statistica tra features e target si indebolisce progressivamente fino al punto medio di corruzione, per poi rafforzarsi nuovamente man mano che il rumore diventa così consistente da stabilire una nuova relazione invertita. Tale pattern spiega l'andamento osservato nelle performance dei modelli, che tendono a performare peggio quando la relazione features-target è statisticamente ambigua (attorno al 50% di rumore).

Conclusioni

Tra i tre modelli esaminati, la **Neural Network** si è dimostrata complessivamente la più solida e resiliente. Nelle prove con valori nulli, ha mantenuto performance stabili anche a livelli estremi di degradazione (fino al 90%), grazie alla sua capacità di apprendere pattern complessi e di gestire imputazioni semplici come mediana o moda senza oscillazioni significative. Con i duplicati, ha mostrato un comportamento costante, non beneficiando né soffrendo eccessivamente della ridondanza, a differenza di modelli come il **Decision Tree naive** che tendeva all'overfitting. Negli outliers, ha registrato metriche affidabili, evitando i crolli osservati nella **SVM**. Infine, di fronte al rumore sulla variabile target, la Neural Network ha mostrato un peggioramento delle performance in linea con quello degli altri modelli analizzati. Questo suggerisce che non si tratti di una debolezza intrinseca del modello, quanto piuttosto di un rumore troppo rilevante per poter essere trascurato. Questa superiorità può essere attribuita alla natura distribuita dell'apprendimento nelle reti neurali, che le rende meno sensibili a perturbazioni locali rispetto a modelli basati su confini decisionali rigidi (come SVM) o regole gerarchiche (come DT).

L'**ottimizzazione** gioca un ruolo cruciale: versioni ottimizzate di tutti i modelli hanno mostrato andamenti più stabili, confermando che una buona tuning degli iperparametri funge da "scudo" contro il rumore, riducendo il rischio di overfitting o underfitting.

Per quanto riguarda il tipo di degradazione più problematico, il **rumore sulla feature target** emerge come il più insidioso e dannoso per tutti i modelli. A differenza di valori nulli (mitigabili con imputazione), duplicati (che possono persino migliorare la stabilità in alcuni casi, come nella

SVM ottimizzata) o outliers (che influenzano principalmente feature continue e modelli sensibili come SVM), il rumore sulla target altera direttamente la relazione fondamentale tra input e output, rendendo l'apprendimento supervisionato intrinsecamente ambiguo. I risultati mostrano un degrado progressivo delle metriche, con un picco negativo di performance registrato intorno al 50% di corruzione, dove la mutua informazione raggiunge il punto più basso e i modelli faticano a discernere pattern coerenti. Oltre questa soglia, si osserva un “miglioramento”, ma solo perché i modelli iniziano a imparare una relazione invertita. Questo tipo di rumore simula errori reali come etichettature sbagliate in dataset medici e sottolinea l'importanza di investire in data labeling di qualità fin dall'inizio.

In aggiunta, è importante osservare che sul **dataset pulito** la rete neurale, pur avendo registrato buone performance, **non è risultata il miglior modello in assoluto**, venendo in alcuni casi superata da algoritmi più leggeri come SVM o Decision Tree ottimizzati. Questo evidenzia come, in assenza di rumore o degradazione dei dati, modelli meno complessi possano risultare più efficienti ed efficaci, offrendo un compromesso migliore tra accuratezza e costi computazionali.

Tuttavia, quando i dati sono stati degradati, la rete neurale ha mostrato una **maggiore robustezza rispetto agli altri modelli**, riuscendo a mantenere performance più stabili. Questo rafforza l'idea che la sua forza emerga soprattutto in contesti reali, dove i dataset raramente sono perfettamente puliti.

Un ulteriore aspetto da sottolineare riguarda i **tempi di esecuzione**. La rete neurale richiede un investimento computazionale nettamente superiore rispetto a Decision Tree e SVM, sia in termini di training che di ottimizzazione degli iperparametri. Pertanto, sebbene la sua robustezza ne faccia il modello più affidabile in presenza di rumore, **il costo in termini di tempo e risorse deve essere tenuto in considerazione**, soprattutto in applicazioni dove l'efficienza è un requisito critico.

Codice Sorgente

Ulteriori dettagli e risorse complete relative a questo progetto, sono reperibili nella mia pagina Github: https://github.com/andreaspagnolo/architetture_dati. All'interno, è possibile trovare il codice sorgente integrale (inclusi i notebook Jupyter per il preprocessing, le degradazioni del dataset e l'addestramento dei modelli), nonché tutti i grafici generati durante l'analisi, compresi numerosi grafici aggiuntivi non incluse nella relazione per motivi di spazio, come distribuzioni dettagliate per singola feature, matrici di confusione complete e andamenti metriche per ogni step di degradazione.